DEVELOPER HANDBOOK

Image Compressor

Technical Documentation

Architecture, logic, algorithms, and worked examples

Field	Value
Project	Image Compressor Android app
Technology	Kotlin, Jetpack Compose, Room, Coroutines, Flow, Coil
Platform	Android 8.0+ (minSdk 26), targetSdk 37
Document scope	Implemented source code in this workspace
Document date	June 2, 2026

Privacy model: All compression work is performed locally on the device. The app does not upload selected images to a server.

How to Read This Handbook

This handbook explains the implemented application as a working system. Each logic section includes the goal, execution steps, a concrete example, important caveats, and direct source-file references. The examples are intentionally numeric where possible so that the behavior can be checked by hand.

> **Notation:** Source references use repository-relative paths followed by line numbers, for example util/ImageCompressor.kt:145. Byte examples use 1 KB = 1024 bytes and 1 MB = 1024 KB.

Contents

1. Project Overview

Image Compressor is a privacy-first Android application for selecting one or more images, previewing their metadata, applying local compression settings, comparing the results, and saving or sharing the compressed files. The app is deliberately serverless: input images stay on the device.

1.1 Functional Coverage

Area	Behavior	Status
Photo selection	Single and multi-select through Android Photo Picker	Implemented
Preview	Grid cards with thumbnail, file size, resolution, and format	Implemented
Compression controls	Quality, target size, resize mode, aspect ratio, JPEG/PNG/WEBP	Implemented
Local processing	Coroutine-based bitmap work on Dispatchers.IO	Implemented
Results	Original/compressed previews, sizes, reduction percentage	Implemented
Export	Gallery save and Android share sheet	Implemented
History	Room records with timestamp, sizes, format, dimensions, and saved URI	Implemented
Privacy onboarding	First-run local-processing explanation	Implemented
Themes	System, light, dark, and Android 12+ dynamic color	Implemented

Source reference: README.md; ui/ImageCompressorApp.kt; util/ImageCompressor.kt

2. Architecture and Runtime Flow

The implementation follows MVVM with a Repository pattern. Compose renders immutable UI state. The ViewModel owns transitions and launches coroutines. The repository coordinates image utilities and Room. Utilities isolate Android bitmap, MediaStore, and share-intent details.

2.1 Layer Responsibilities

Layer	Responsibility	Primary source
Compose UI	Renders screens and dispatches user actions	ui/ImageCompressorApp.kt
ViewModel	Owns screen state, progress, messages, and orchestration	ui/ImageCompressorViewModel.kt
Repository	Coordinates inspection, compression, save, and Room history	data/ImageRepository.kt
Compression utility	Decodes, resizes, rotates, encodes, and caches images	util/ImageCompressor.kt
Storage utilities	Writes MediaStore files and launches share intents	util/GallerySaver.kt; util/ShareUtils.kt
Local persistence	Room database and SharedPreferences-backed flows	data/local/*; data/preferences/*
Dependency container	Builds and shares application-scoped dependencies	ImageCompressorApplication.kt

2.2 End-to-End Flow

1. The user completes onboarding and opens the Home screen.
2. The Photo Picker returns one or more content URIs.
3. The repository attempts to retain URI read access and inspects metadata off the main thread.
4. The user chooses quality, target size, resize mode, aspect-ratio behavior, and output format.
5. The repository compresses each image sequentially on Dispatchers.IO and inserts a Room history row.
6. The ViewModel updates progress after each completed image.
7. The Results screen compares before and after values and offers save or share actions.
8. Saving writes to MediaStore on Android 10+ or the public Pictures directory on Android 8/9.

Example flow: A user selects three photos, chooses 80% JPEG quality and a 500 KB target, then taps Compress Images. The progress state moves through 1/3, 2/3, and 3/3. Each output is cached, recorded in Room, shown in the Results screen, and saved only if the user taps Save to Gallery.

Source reference: ui/ImageCompressorApp.kt:114-232; ui/ImageCompressorViewModel.kt:101-215; data/ImageRepository.kt:43-81

3. Core Data Models and Derived Values

3.1 Domain Models

Model	Purpose
CompressionSettings	User-selected quality, target bytes, resize mode, dimensions, aspect-ratio toggle, format
SelectedImage	Input URI plus inspected display name, bytes, resolution, and format
CompressedImage	Output URI/path, output bytes, dimensions, format, history row ID, optional saved URI
CompressionHistoryEntity	Room record for a completed compression result
ImageCompressorUiState	Single immutable snapshot consumed by the Compose UI

3.2. Reduction Percentage

Purpose. Convert the before/after byte counts into a beginner-friendly percentage shown in Results and History.

Algorithm.

1. If the original byte count is zero or unknown, return 0 to avoid division by zero.
2. Compute 1 - (compressedBytes / originalBytes).
3. Multiply by 100, round to the nearest integer, and clamp to the range 0..100.

4. Clamping prevents a larger output file from displaying a negative reduction.

> **Worked example:** For a 4 MB original and a 1 MB output: (1 - 1/4) * 100 = 75%. For a 100 KB original and a 150 KB output, the raw value is -50%, but the displayed value is clamped to 0%.

Source reference: data/model/CompressionModels.kt:61-66; ui/ImageCompressorApp.kt:695-703

3.3. Readable File Sizes

Purpose. Format raw byte counts into readable KB or MB labels for the UI.

Algorithm.

1. Return 'Unknown size' for non-positive values.
2. Divide bytes by 1024 to obtain KB.
3. If KB is less than 1024, show one decimal place.
4. Otherwise divide KB by 1024 and show MB with two decimal places.

> **Worked example:** 1024 bytes becomes 1.0 KB. 1,048,576 bytes becomes 1.00 MB. A provider that reports 0 bytes is displayed as Unknown size.

Source reference: data/model/CompressionModels.kt:68-73; data/model/CompressionModelsTest.kt:14-18

4. Image Selection and Metadata Inspection

4.1. Photo Picker Launch

Purpose. Select only the images the user explicitly chooses, without requesting broad media-library read access.

Algorithm.

1. Register PickMultipleVisualMedia with a maximum of 30 images and PickVisualMedia for a single image.
2. Launch each picker with ImageOnly so videos are excluded.
3. Pass returned URIs to the ViewModel.
4. Use AndroidX Photo Picker behavior so older supported devices can fall back to the system document picker.

> **Worked example:** On Home, Select Images launches the multi-picker. In Preview, Add One launches the single-picker and Add Multiple launches the multi-picker. Selecting two new photos sends a two-URI list to addSelectedImages().

Source reference: ui/ImageCompressorApp.kt:114-146; AndroidManifest.xml:29-41

4.2. URI Retention and De-duplication

Purpose. Retain read access where the URI provider supports it and avoid duplicate cards when the same image is selected twice.

Algorithm.

1. Call distinct() on the incoming URI list before inspecting it.
2. Attempt takePersistableUriPermission() with read access inside runCatching.
3. Continue even if a provider does not support persistable grants.
4. After inspection, merge old and new selections and call distinctBy { id }, where id is uri.toString().

Worked example: If the existing selection contains content://photos/42 and the next picker response contains content://photos/42 plus content://photos/99, only two cards remain: 42 and 99.

Engineering note: Persistable URI grants are best-effort because Photo Picker providers and document providers do not all expose identical grant behavior.

Source reference: data/ImageRepository.kt:43-54; data/model/CompressionModels.kt:35-44; ui/ImageCompressorViewModel.kt:101-126

4.3. Metadata Query

Purpose. Show useful metadata before compression without decoding a full-resolution bitmap into memory.

Algorithm.

1. Query OpenableColumns.DISPLAY_NAME and OpenableColumns.SIZE through ContentResolver.
2. If the reported size is missing or non-positive, ask openAssetFileDescriptor() for the asset length.
3. Decode bounds only to obtain pixel width and height.
4. Read EXIF orientation and swap displayed width/height for 90-degree or 270-degree rotation.
5. Resolve the MIME type and show its subtype in uppercase.

Worked example: A provider reports vacation.jpg as 3,145,728 bytes with encoded bounds 4000 x 3000 and EXIF rotation 90. The preview card shows 3.00 MB, 3000 x 4000, and JPEG.

Source reference: util/ImageCompressor.kt:35-51; util/ImageCompressor.kt:114-143; util/ImageCompressor.kt:169-185

4.4. Bounds-Only Decode

Purpose. Read image dimensions safely before allocating pixel memory.

Algorithm.

1. Create BitmapFactory.Options with inJustDecodeBounds = true.
2. Open the content URI stream and fail clearly if the provider cannot open it.
3. Call BitmapFactory.decodeStream(). In bounds-only mode, its return value is intentionally ignored.
4. Validate options.outWidth and options.outHeight after decoding.

Worked example: For an 8000 x 6000 photo, decodeStream() returns no bitmap because inJustDecodeBounds is true, but options.outWidth becomes 8000 and options.outHeight becomes 6000. The app uses those options values.

Engineering note: This distinction matters: treating the expected null bitmap return as an error would reject valid images. The implementation explicitly checks stream availability separately from populated bounds.

Source reference: util/ImageCompressor.kt:134-143

5. Resize, Sampling, and Memory Protection

5.1. Percentage Resize

Purpose. Scale both dimensions by the user's chosen percentage while preserving the image proportions.

Algorithm.

1. Clamp the percentage to 10..100.
2. Convert it into a decimal scale by dividing by 100.
3. Multiply width and height by that scale.
4. Round each value and ensure each dimension is at least 1 pixel.

Worked example: A 4000 x 3000 image resized to 50% becomes 2000 x 1500. A 10% resize becomes 400 x 300.

Source reference: util/ImageCompressor.kt:202-208; ui/ImageCompressorViewModel.kt:150-152

5.2. Custom Resize With Aspect Ratio

Purpose. Fit the original image inside a custom width/height box without stretching it.

Algorithm.

1. Parse the width and height text fields. If a value is missing, fall back to the source dimension.
2. When aspect-ratio preservation is enabled, compute widthScale = requestedWidth / sourceWidth.
3. Compute heightScale = requestedHeight / sourceHeight.
4. Use the smaller scale so both output dimensions fit inside the requested box.
5. When preservation is disabled, use the requested dimensions directly.

Worked example: A 4000 x 3000 image placed into a 1000 x 1000 box has widthScale = 0.25 and heightScale = 0.333. The smaller scale is 0.25, so the result is 1000 x 750. With preservation disabled, it becomes 1000 x 1000.

Source reference: util/ImageCompressor.kt:209-219; ui/ImageCompressorApp.kt:518-544

5.3. Pixel Budget Constraint

Purpose. Prevent unusually large requested dimensions from creating oversized in-memory bitmaps.

Algorithm.

1. Calculate pixels = width * height.

2. If pixels is at most 16,000,000, keep the dimensions.
3. Otherwise compute scale = sqrt(16,000,000 / pixels).
4. Multiply both dimensions by that scale to preserve proportions while approaching the pixel limit.

Worked example: An 8000 x 6000 request contains 48,000,000 pixels. scale = sqrt(16,000,000 / 48,000,000) = 0.577. The constrained dimensions are approximately 4619 x 3464, close to 16 million pixels.

Source reference: util/ImageCompressor.kt:220-230; util/ImageCompressor.kt:270-272

5.4. Sampled Bitmap Decode

Purpose. Reduce memory usage by decoding an appropriately sampled bitmap instead of always loading the full image.

Algorithm.

1. Read source bounds first.
2. Start inSampleSize at 1.
3. Double the sample size while the next downsampled width and height are still at least the requested dimensions.
4. Continue doubling if the decoded pixel count would remain above the 16-million-pixel memory budget.
5. Decode the URI stream using the chosen inSampleSize and ARGB_8888.

Worked example: For an 8000 x 6000 image requested at 2000 x 1500, sample size progresses 1 -> 2 -> 4. A further jump to 8 would decode only 1000 x 750, below the request, so the selected sample is 4.

Engineering note: Sampling is power-of-two based because BitmapFactory.inSampleSize is designed around efficient decoder sampling.

Source reference: util/ImageCompressor.kt:145-167

5.5. EXIF-Aware Rotation

Purpose. Keep compressed output visually upright when the camera stored orientation as metadata rather than rotated pixels.

Algorithm.

1. Read EXIF TAG_ORIENTATION from the selected URI stream.
2. Map ROTATE_90, ROTATE_180, and ROTATE_270 to degree values.
3. Create a Matrix rotation during bitmap decode.
4. Recycle the pre-rotation bitmap if a new rotated bitmap is created.
5. Write ORIENTATION_NORMAL to the exported file because its pixels are already upright.

Worked example: A portrait photo may be stored as 4032 x 3024 pixels plus ROTATE_90 metadata. The compressor rotates the pixels and exports an upright 3024 x 4032 image marked ORIENTATION_NORMAL.

Engineering note: The implementation handles rotational EXIF values. Mirrored EXIF orientations are not transformed.

Source reference: util/ImageCompressor.kt:90-96; util/ImageCompressor.kt:169-193

6. Compression and Output Algorithms

6.1. Main Compression Pipeline

Purpose. Convert a SelectedImage and CompressionSettings object into a cached CompressedImage result.

Algorithm.

1. Calculate requested output dimensions.
2. Decode a sampled bitmap and apply EXIF rotation.
3. Scale the decoded bitmap to the final requested dimensions.
4. Flatten transparency onto white when exporting JPEG.
5. Encode to bytes at the chosen quality.
6. Apply target-size iteration when enabled.
7. Write a uniquely named cache file, normalize EXIF orientation, recycle the bitmap, and return output metadata.

Worked example: A 4000 x 3000 PNG is configured for 50% resize and JPEG output at 80 quality. It is decoded efficiently, scaled to 2000 x 1500, flattened onto white if it has alpha, JPEG encoded, and written into cache/compressed/.

Source reference: util/ImageCompressor.kt:53-112

6.2. JPEG Transparency Flattening

Purpose. Avoid unexpected dark or transparent backgrounds when an alpha-enabled input is converted to JPEG.

Algorithm.

1. Check whether the requested output format is JPEG and whether the bitmap reports alpha.
2. Create an ARGB_8888 destination bitmap with matching dimensions.
3. Fill its canvas with white.
4. Draw the original bitmap over the white background.
5. Recycle the old bitmap and encode the flattened result.

Worked example: A transparent PNG logo converted to JPEG gets a white background. Without flattening, transparent pixels could be rendered unpredictably by the encoder or downstream viewer.

Source reference: util/ImageCompressor.kt:239-248

6.3. Format-Specific Encoding

Purpose. Map the user's output choice to Android bitmap encoders while accounting for platform differences.

Algorithm.

1. Use Bitmap.CompressFormat.JPEG for JPEG output.
2. Use Bitmap.CompressFormat.PNG for PNG output.
3. Use WEBP_LOSSY on Android 11+ and the older WEBP constant on earlier supported versions.
4. Write encoded bytes into a ByteArrayOutputStream and fail if bitmap.compress() returns false.

Worked example: On Android 13, WEBP uses WEBP_LOSSY. On Android 9, the app falls back to the legacy WEBP encoder constant. PNG remains lossless, so the quality slider has less influence on PNG file size.

Source reference: util/ImageCompressor.kt:250-263; data/model/CompressionModels.kt:6-10

6.4. Target File Size Iteration

Purpose. Try to bring the output close to an optional target size without infinite looping.

Algorithm.

1. Encode once using the configured quality.
2. While encoded bytes exceed the target and attempts are below 40, continue adjusting.
3. For JPEG or WEBP with quality above 10, reduce quality by 5 and encode again.
4. For PNG, or after lossy quality reaches 10, reduce bitmap width and height to 90% of the prior values.
5. Stop when bytes are within target, dimensions no longer change, or 40 attempts have run.

Worked example: A JPEG initially encodes to 820 KB at quality 80 with a 500 KB target. The loop retries at qualities 75, 70, 65, and so on. If quality 10 is still too large, dimensions shrink to 90% and encoding continues.

Engineering note: The target is best-effort. Some images cannot reach the requested size without excessive dimension loss, and the 40-attempt cap prevents unbounded work.

Source reference: util/ImageCompressor.kt:63-81; util/ImageCompressor.kt:265-271

6.5. Unique Cache Output and FileProvider URI

Purpose. Create shareable temporary result files without exposing raw filesystem paths to other apps.

Algorithm.

1. Ensure cacheDir/compressed exists.
2. Build a filename from a fixed prefix, current epoch milliseconds, an eight-character UUID suffix, and format extension.
3. Write the encoded bytes to that file.
4. Create a content URI using FileProvider and the app-specific authority.
5. Return both the private file path for internal copying and the content URI for previews or sharing.

Worked example: One JPEG output might be named ImageCompressor_1780412812345_a1b2c3d4.jpg. Another compression in the same session receives a different timestamp or UUID suffix, avoiding collisions.

Source reference: util/ImageCompressor.kt:83-112; AndroidManifest.xml:19-27; res/xml/file_paths.xml:1-6

7. Batch Processing, Progress, and Persistence

7.1. Sequential Batch Compression

Purpose. Compress multiple images off the main thread while producing predictable progress updates and Room records.

Algorithm.

1. Move repository work into withContext(Dispatchers.IO).
2. Iterate through selected images with mapIndexed.
3. Compress one image.
4. Insert its completed result into Room and capture the generated history ID.
5. Invoke onProgress(index + 1, total).
6. Return the CompressedImage with its history ID attached.

Worked example: For three selected photos, the repository emits progress after each output: completed=1,total=3; completed=2,total=3; completed=3,total=3. The UI fractions are 0.333, 0.667, and 1.0.

Engineering note: Compression is sequential rather than parallel. This is conservative for memory use because several large bitmaps are not decoded at the same time.

Source reference: data/ImageRepository.kt:56-81; ui/ImageCompressorViewModel.kt:174-215

7.2. Progress Fraction

Purpose. Convert completed and total counts into a value suitable for Material 3 LinearProgressIndicator.

Algorithm.

1. If total is zero, return 0f.

2. Otherwise divide completed.toFloat() by total.
3. Pass the fraction to the Results loading card while isCompressing is true.

Worked example: During the second image of a four-image batch, completed=2 and total=4. fraction = 2 / 4 = 0.5, so the progress bar displays 50%.

Source reference: ui/ImageCompressorViewModel.kt:32-35; ui/ImageCompressorApp.kt:577-586

7.3. Room History Flow

Purpose. Persist completed compression summaries locally and keep the History UI reactive.

Algorithm.

1. Store each completion as CompressionHistoryEntity in the compression_history table.
2. Observe rows ordered by createdAt descending as Flow<List<CompressionHistoryEntity>>.
3. Collect the Flow in the ViewModel and copy the latest list into UI state.
4. Update savedPath after a successful gallery save.
5. Support deleting one row or clearing all rows.

Worked example: After compressing beach.jpg, Room stores originalSizeBytes=3145728 and compressedSizeBytes=524288. When the user saves it, updateSavedPath() adds the returned MediaStore content URI. History refreshes automatically.

Source reference: data/local/CompressionHistoryEntity.kt:6-19; data/local/CompressionHistoryDao.kt:9-23; data/ImageRepository.kt:83-95

8. Gallery Save, Sharing, and Android Versions

8.1. Scoped MediaStore Save on Android 10+

Purpose. Publish a compressed file to the gallery without requesting legacy broad storage permission.

Algorithm.

1. Insert an Images.Media row with display name, MIME type, Pictures/Image Compressor relative path, and IS_PENDING=1.
2. Open the destination output stream and copy the private cache file.
3. Clear the pending marker by updating IS_PENDING=0.
4. If copying fails, delete the incomplete MediaStore row and rethrow the error.

Worked example: On Android 14, saving ImageCompressor_...jpg inserts a row under Pictures/Image Compressor, copies bytes, then makes the row visible by clearing IS_PENDING. No runtime storage prompt is shown.

Source reference: util/GallerySaver.kt:22-56

8.2. Legacy Save on Android 8 and 9

Purpose. Support API 26-28 devices that predate scoped storage.

Algorithm.

1. Check WRITE_EXTERNAL_STORAGE permission before writing.
2. Create the public Pictures/Image Compressor directory.
3. Copy the cache file into the public directory.
4. Ask MediaScannerConnection to index the file.
5. Return the indexed URI, or a file URI fallback if scanning returns null.

Worked example: On Android 9, the first save triggers the runtime storage prompt. After approval, the cache JPEG is copied to Pictures/Image Compressor and scanned so gallery apps can display it.

Source reference: ui/ImageCompressorApp.kt:122-140; util/GallerySaver.kt:58-82; AndroidManifest.xml:5-7

8.3. Single and Multi-Image Sharing

Purpose. Send cached compressed outputs to other apps through the Android share sheet.

Algorithm.

1. Return immediately when the image list is empty.
2. For one URI, create ACTION_SEND with the exact image MIME type.
3. For multiple URIs, create ACTION_SEND_MULTIPLE with image/*.
4. Attach EXTRA_STREAM values and ClipData entries.
5. Grant temporary read access with FLAG_GRANT_READ_URI_PERMISSION.
6. Launch an Android chooser.

Worked example: Sharing one WEBP uses ACTION_SEND and image/webp. Sharing three mixed compressed images uses ACTION_SEND_MULTIPLE and image/* with three stream URIs.

Source reference: util/ShareUtils.kt:9-42; ui/ImageCompressorApp.kt:202-223

9. UI State, Navigation, and Theme Logic

9.1. Single Immutable UI State

Purpose. Keep the Compose interface consistent by rendering one state object rather than many unrelated mutable fields.

Algorithm.

1. Initialize a MutableStateFlow<ImageCompressorUiState> in the ViewModel.
2. Expose it as read-only StateFlow.
3. Update state with copy() so each transition is explicit.
4. Collect state with collectAsStateWithLifecycle() in the root composable.
5. Render exactly one screen based on state.screen.

Worked example: When compression starts, one copy() call sets screen=RESULTS, clears old outputs, sets total, and marks isCompressing=true. The Results screen immediately renders the progress card.

Source reference: ui/ImageCompressorViewModel.kt:37-76; ui/ImageCompressorViewModel.kt:180-189; ui/ImageCompressorApp.kt:89-103

9.2. Screen Navigation and Back Mapping

Purpose. Provide predictable beginner-friendly navigation without a separate navigation framework.

Algorithm.

1. Represent screens with AppScreen enum values.
2. Update screen directly for top-level navigation.
3. Map Back from Preview to Home, Settings to Preview, Results to Settings, and History/Settings to Home.
4. Enable BackHandler for non-root screens.
5. Show Home, History, and Settings in the bottom navigation bar.

Worked example: A user on Results taps Back. The ViewModel maps RESULTS to COMPRESSION_SETTINGS so the user can adjust quality without reselecting photos.

Source reference: ui/ImageCompressorViewModel.kt:22-30; ui/ImageCompressorViewModel.kt:83-99; ui/ImageCompressorApp.kt:155-175

9.3. Onboarding and Theme Preferences

Purpose. Remember first-run completion and the user's chosen appearance across launches.

Algorithm.

1. Read onboarding_complete and theme_preference from private SharedPreferences.
2. Wrap both values in StateFlow.
3. Start on ONBOARDING when onboarding_complete is false; otherwise start on HOME.
4. Update both SharedPreferences and the matching flow when a preference changes.
5. Resolve SYSTEM, LIGHT, or DARK into a boolean darkTheme value at the Compose root.
6. Use Android 12+ dynamic color when available, otherwise fall back to app light/dark schemes.

Worked example: A first launch sees onboarding. After Get Started, onboarding_complete is saved. If the user later chooses Dark, the next launch restores DARK and the root theme renders dark colors immediately.

Source reference: data/preferences/AppPreferencesRepository.kt:10-51; ui/ImageCompressorViewModel.kt:56-81; ui/ImageCompressorApp.kt:89-104; theme/Theme.kt:27-45

9.4. Responsive Image Grid and UI Statuses

Purpose. Remain usable across phone widths while clearly showing empty, loading, success, and error states.

Algorithm.

1. Use GridCells.Adaptive(160.dp) so Compose chooses the number of preview columns that fit.
2. Show LoadingCard while image metadata is loading or compression is running.
3. Show EmptyState when no selection, no results, or no history exists.
4. Use SnackbarHostState to present messages, then consume each message after display.
5. Disable already-saved result buttons and change the label to Saved.

> **Worked example:** A narrow phone may fit one 160 dp image card per row; a tablet may fit several. While metadata loads, the grid is replaced with Reading image details… and a progress indicator.

Source reference: ui/ImageCompressorApp.kt:148-153; ui/ImageCompressorApp.kt:351-418; ui/ImageCompressorApp.kt:567-654; ui/ImageCompressorApp.kt:752-789

10. Privacy, Failure Handling, and Constraints

10.1 Privacy and Permission Matrix

Action	Android version	Mechanism	Permission behavior
Select images	Android 8.0+	Photo Picker or document-picker fallback	No broad gallery-read permission
Compress images	Android 8.0+	Private cache files	No runtime permission
Save output	Android 10+	Scoped MediaStore	No runtime storage permission
Save output	Android 8/9	Public Pictures directory	WRITE_EXTERNAL_STORAGE requested at save time
Share output	Android 8.0+	FileProvider content URIs	Temporary URI read grant

The manifest disables backups and excludes root data from cloud backup and device transfer. The privacy onboarding and Settings screen state that images are compressed locally.

Source reference: AndroidManifest.xml:5-41; res/xml/data_extraction_rules.xml:1-9; ui/ImageCompressorApp.kt:278-305; ui/ImageCompressorApp.kt:739-743

10.2 Failure Handling

Condition	Message	Behavior
URI cannot open	Unable to open the selected image.	Inspection fails and snackbar receives the error.

Condition	Message	Behavior
Format bounds unavailable	This image format could not be decoded.	Reject unsupported or invalid source.
Bitmap decode fails	Unable to decode the selected image.	Compression stops for the current batch.
Encode returns false	Unable to encode the compressed image.	No cache file is returned.
MediaStore insert fails	Unable to create a gallery entry.	Save action fails clearly.
Legacy permission denied	Storage permission was not granted.	Image remains available in Results but is not saved.

Source reference: util/ImageCompressor.kt:134-165; util/ImageCompressor.kt:258-262; util/GallerySaver.kt:41-55; ui/ImageCompressorApp.kt:122-140

10.3 Known Constraints and Extension Notes

- Target-size output is best-effort rather than an exact byte guarantee.
- Sequential batch processing favors predictable memory use over maximum CPU throughput.
- Temporary compressed files live under app cache. Android may evict cache files later; saved gallery copies are durable.
- EXIF rotational orientation is handled, but mirrored EXIF orientations are not transformed.
- History stores compressed cache URIs and optional gallery URIs. Sharing a very old unsaved cache URI may fail after cache eviction.
- Room schema version is 1. Future schema changes should add explicit migrations before release upgrades.
- Release minification is disabled. Production publishing can enable R8 after validating sharing, Room, and Compose behavior.

11. Build, Dependencies, and Verification

11.1 Build Configuration

Concern	Configuration
Language	Kotlin with Java 17 toolchain
UI	Jetpack Compose with Material 3
SDK	compileSdk 37, targetSdk 37, minSdk 26
Database	Room runtime, Room KTX, KSP compiler, exported schema
Image preview	Coil Compose
Metadata	AndroidX ExifInterface

Concern	Configuration
Concurrency	Kotlin Coroutines and Flow through lifecycle/viewmodel dependencies

Source reference: app/build.gradle.kts:1-91; gradle/libs.versions.toml

11.2 Automated Checks

Run the following commands from the project root:

```
./gradlew testDebugUnitTest assembleDebug
./gradlew lintDebug
```

Function	Input	Expected result
calculateReductionPercent	1000 -> 250 bytes	75%
calculateReductionPercent	0 -> 250 bytes	0% without division by zero
calculateReductionPercent	100 -> 150 bytes	0% after negative clamp
toReadableSize	1024 bytes	1.0 KB
toReadableSize	1,048,576 bytes	1.00 MB

Source reference: data/model/CompressionModelsTest.kt:1-19; README.md

11.3 Manual Verification Checklist

- Install the debug APK and complete onboarding.
- Pick one portrait camera image and confirm its displayed orientation and resolution.
- Pick several images and confirm duplicate selections are not repeated.
- Compress JPEG at multiple quality values and compare output byte sizes.
- Compress with each target-size preset and observe best-effort results.
- Test percentage resize and custom resize with aspect-ratio preservation on and off.
- Convert a transparent PNG to JPEG and confirm the background becomes white.
- Save on Android 10+ and verify the Pictures/Image Compressor album.
- Save on Android 8/9 and verify the storage prompt occurs only at save time.
- Share one result and several results through the Android share sheet.
- Open History, delete one item, clear all items, and switch system/light/dark themes.

Appendix A. Pseudocode Reference

A.1 Compression Pipeline

```
compress(image, settings):
    dimensions = calculateOutputDimensions(image, settings)
    bitmap = decodeSampledBitmap(image.uri, dimensions)
    bitmap = scaleBitmap(bitmap, dimensions)
    bitmap = flattenTransparencyForJpeg(bitmap, settings.format)
    bytes = encode(bitmap, settings.format, settings.quality)
    bytes, bitmap = approachTargetSize(bytes, bitmap, settings)
    file = writeUniqueCacheFile(bytes, settings.format)
    normalizeExifOrientation(file)
    recycle(bitmap)
    return CompressedImage(fileProviderUri(file), file.path, file.length, dimensions)
```

A.2 Target-Size Loop

```
attempts = 0
while target exists and encodedBytes > target and attempts < 40:
    if format is lossy and quality > 10:
        quality = max(10, quality - 5)
    else:
        bitmap = scale(bitmap, width * 0.9, height * 0.9)
    encodedBytes = encode(bitmap, format, quality)
    attempts += 1
```

A.3 Pixel-Budget Constraint

```
pixels = width * height
if pixels <= 16_000_000:
    return width, height
scale = sqrt(16_000_000 / pixels)
return round(width * scale), round(height * scale)
```

A.4 MediaStore Save

```
if Android >= 10:
    uri = insert MediaStore row with IS_PENDING = 1
    copy cache file into uri output stream
    update row with IS_PENDING = 0
else:
    require WRITE_EXTERNAL_STORAGE
    copy into Pictures/Image Compressor
    scan file with MediaScannerConnection
```

Appendix B. Source-File Ownership Map

Source file	Ownership
MainActivity.kt	Compose entry point and edge-to-edge activity setup
ImageCompressorApplication.kt	Application-scoped container and dependency construction
ui/ImageCompressorApp.kt	All Compose screens, Photo Picker, save permission launcher, status UI
ui/ImageCompressorViewModel.kt	MVVM state, actions, progress, save/history orchestration
data/model/CompressionModels.kt	Domain models, enums, reduction and readable-size helpers
data/ImageRepository.kt	Repository interface and implementation for batch coordination
data/local/*	Room entity, DAO, and database
data/preferences/*	Onboarding and theme SharedPreferences flows
util/ImageCompressor.kt	Bitmap inspection, resizing, sampling, EXIF handling, encoding, cache output
util/GallerySaver.kt	Android-version-specific gallery publishing
util/ShareUtils.kt	Single and multi-image Android sharing
AndroidManifest.xml	Legacy save permission, FileProvider, Photo Picker backport hook
res/xml/file_paths.xml	FileProvider cache-path exposure
res/xml/data_extraction_rules.xml	Backup and device-transfer exclusions
app/build.gradle.kts	SDK levels, plugins, and dependencies
data/model/CompressionModelsTest.kt	Unit tests for derived size math

Appendix C. Practical Extension Playbook

Change	Implementation direction
Add HEIC output	Extend OutputFormat, confirm Bitmap encoder support, update MIME mapping and UI chips.
Parallelize batches	Use bounded concurrency and a semaphore; measure heap behavior before increasing parallelism.
Exact target sizing	Replace linear quality steps with a bounded binary search, then dimension fallback.
Retain durable output	Move unsaved history outputs from cache to app files storage or add explicit export policy.
Add Room schema fields	Increment database version and ship a migration instead of relying on destructive fallback.
Add mirrored EXIF support	Map flip and transpose EXIF values into Matrix transforms before export.

Closing note: This handbook describes the implemented project as of June 2, 2026. Keep it versioned with the source code and update the algorithm examples whenever compression constants, permissions, or persistence behavior change.